SeatRush Runbook

Every command to run, and the result you should see

How to use this document: work top to bottom. Each step has a command (grey box) and the expected result (green box). **Stop at the first step that does not match** and check the 'If it does not match' note. Values such as ids, timestamps and payment references will differ on your run; the shape and the status codes should match.

Part 0: Setup (do once)

Open Windows PowerShell and paste this. It sets your site address and defines a helper called **Call** that prints the HTTP status code plus the body, which is useful for seeing 403 / 409 responses that Invoke-RestMethod would otherwise hide behind an error.

RUN THIS (PowerShell)

```
$u = "https://seatrush.onrender.com"     # replace with your own Render URL

function Call($method, $path, $headers=@{}, $body=$null) {
  $p = @{ Method=$method; Uri="$u$path"; Headers=$headers; UseBasicParsing=$true }
  if ($body) { $p.Body=$body; $p.ContentType="application/json" }
  try   { $r = Invoke-WebRequest @p; "$($r.StatusCode)  $($r.Content)" }
  catch { "$([int]$_.Exception.Response.StatusCode)  $($_.ErrorDetails.Message)" }
}
```

EXPECTED RESULT

```
(no output; the prompt just returns)
```

Cold start warning: the free Render service sleeps after about 15 minutes idle. The first request can take up to a minute. Run Step 1 once and wait for it before judging anything.

Common mistake: /health is at the site root. The address `/docs/health` does not exist and returns {"detail":"Not Found"}. The site root `/` also returns Not Found; the entry page is `/docs`.

Part 1: Is it alive and connected?

Step 1: Server is up

Confirms the web service is running.

RUN THIS (PowerShell)

```
Call GET "/health"
```

EXPECTED RESULT

```
200  {"status":"ok"}
```

If it does not match: A timeout or 502 usually means the service is still waking up or the deploy failed. Wait one minute, retry, then open the service Logs tab in Render.

Step 2: Database connected and seeded (Neon)

The app creates the tables and 2 events on first boot.

RUN THIS (PowerShell)

```
Call GET "/events"
```

EXPECTED RESULT

```
200  [{"id":1,"name":"Open Sale Concert","venue":"City Hall","starts_at":"...","waiting_room":false},
      {"id":2,"name":"Mega Tour Flash Sale","venue":"Stadium","starts_at":"...","waiting_room":true}]
```

If it does not match: A 500 error means the Neon connection failed. Check DATABASE_URL (pooled string) and DIRECT_DATABASE_URL (direct string) in Render, then read the Logs.

Step 3: Seats exist

A fresh database has 2000 seats per event.

RUN THIS (PowerShell)
```
Call GET "/events/1/availability"
```

EXPECTED RESULT
```
200  {"available":2000,"held":0,"sold":0}
```

If it does not match: Numbers differ if you already ran tests. That is fine. All zeros means seeding did not run (check AUTO_SEED=1).

Step 4: Redis connected

Every request passes through the Redis rate limiter and the cache, so a good response here means Redis works.

RUN THIS (PowerShell)
```
Call GET "/events/1/seats?limit=3" @{"x-user-id"="alice"}
```

EXPECTED RESULT
```
200  {"seats":[{"seat_id":"S000-R00-00","price_cents":15000},{"seat_id":"S000-R00-01","price_cents":15000},
       {"seat_id":"S000-R00-02","price_cents":15000}],"next_after":"S000-R00-02"}
```

If it does not match: A 500 error here (while /events worked) points to REDIS_URL being missing or wrong in Render.

Part 2: Does the core logic work?

These steps run one purchase end to end. Use the same PowerShell window so the variables carry over.

Step 5: Reserve a seat (hold)

Picks the first free seat and holds it for alice for 2 minutes.

RUN THIS (PowerShell)
```
$h    = @{ "x-user-id"="alice" }
$seat = (Invoke-RestMethod "$u/events/1/seats?limit=1" -Headers $h).seats[0].seat_id
$body = (@{ seat_ids=@($seat) } | ConvertTo-Json -Compress)
$hold = Invoke-RestMethod -Method Post -Uri "$u/events/1/holds" -Headers $h `
  -ContentType "application/json" -Body $body
$hold | Format-List
```

EXPECTED RESULT
```
hold_id      : 3f6c1d52-....(a UUID)
event_id     : 1
seat_ids     : {S000-R00-00}
amount_cents : 15000
expires_at   : 2026-10-10T13:05:00+00:00     (about 2 minutes from now)
```

If it does not match: A 429 means you were rate limited; wait a few seconds and retry.

Step 6: Nobody else can take it (no overselling)

A different user asks for the same seat. This is the central guarantee of the project.

RUN THIS (PowerShell)
```
Call POST "/events/1/holds" @{"x-user-id"="bob"} $body
```

EXPECTED RESULT
```
409  {"detail":"one or more seats are unavailable"}
```

If it does not match: If you get 201, stop: that would mean double booking. Re-run Step 5 first, because bob's request must come after alice's hold.

Step 7: Buy the held seat (checkout saga)

Creates an order, charges the simulated payment provider and confirms the seat.

RUN THIS (PowerShell)

```
$oh = @{ "x-user-id"="alice"; "idempotency-key"="order-$(Get-Random)" }
$ob = (@{ hold_id=$hold.hold_id } | ConvertTo-Json -Compress)
$order = Invoke-RestMethod -Method Post -Uri "$u/orders" -Headers $oh `
  -ContentType "application/json" -Body $ob
$order | Format-List
```

EXPECTED RESULT

```
Most of the time (about 90%):
order_id       : 9a41....      status : CONFIRMED     payment_ref : pay_1a2b3c...
seat_ids       : {S000-R00-00} amount_cents : 15000   failure_reason :

About 10% of the time (simulated card decline, this is correct behaviour):
status : FAILED      failure_reason : payment_declined      (the seat is released again)
```

If it does not match: On FAILED, repeat Steps 5 and 7 with a new hold. A 410 'hold expired' means more than 2 minutes passed; redo Step 5.

Step 8: Retrying is safe (idempotency)

Send the exact same order request again with the same idempotency key. No second charge.

RUN THIS (PowerShell)

```
$again = Invoke-RestMethod -Method Post -Uri "$u/orders" -Headers $oh `
  -ContentType "application/json" -Body $ob
$again.order_id -eq $order.order_id
```

EXPECTED RESULT

```
True
```

If it does not match: Run Step 7 first so that $oh, $ob and $order exist in this window.

Step 9: Read the order back

Fetches the stored order by id.

RUN THIS (PowerShell)

```
Invoke-RestMethod "$u/orders/$($order.order_id)" | Format-List
```

EXPECTED RESULT

```
The same order_id, status and seat_ids as in Step 7.
```

Step 10: Waiting room blocks unadmitted users

Event 2 is the flash sale. Holding a seat without being admitted must be refused.

RUN THIS (PowerShell)

```
Call POST "/events/2/holds" @{"x-user-id"="carol"} '{"seat_ids":["S000-R00-01"]}'
```

EXPECTED RESULT

```
403  {"detail":"join the waiting room and wait to be admitted"}
```

Step 11: Join the waiting room

Gets carol a ticket in the first-in-first-out line.

RUN THIS (PowerShell)

```
$c = @{ "x-user-id"="carol" }
$q = Invoke-RestMethod -Method Post -Uri "$u/events/2/queue/join" -Headers $c
$q | Format-List
```

EXPECTED RESULT

```
ticket       : 5be2a8c1d3f04a77:carol
admitted     : False
position     : 1
queue_length : 1          (position and length may be larger if others joined)
```

Step 12: Wait to be admitted

A background loop admits users every second. This polls for up to 30 seconds.

RUN THIS (PowerShell)

```
$t = [uri]::EscapeDataString($q.ticket); $i = 0
do {
  $s = Invoke-RestMethod "$u/events/2/queue/status?ticket=$t" -Headers $c
  "admitted=$($s.admitted) position=$($s.position)"
  if (-not $s.admitted) { Start-Sleep 2 }; $i++
} until ($s.admitted -or $i -ge 15)
```

EXPECTED RESULT

```
admitted=False position=1        (may print once or not at all)
admitted=True position=0
```

If it does not match: Never admitted means the background loops are not running. Make a normal request to wake the service, then check Render Logs for 'admission error'.

Step 13: Hold a seat after admission

Now the ticket is accepted. Send it in the x-queue-ticket header.

RUN THIS (PowerShell)

```
$seat2 = (Invoke-RestMethod "$u/events/2/seats?limit=1" -Headers $c).seats[0].seat_id
$h2 = @{ "x-user-id"="carol"; "x-queue-ticket"=$q.ticket }
$b2 = (@{ seat_ids=@($seat2) } | ConvertTo-Json -Compress)
Invoke-RestMethod -Method Post -Uri "$u/events/2/holds" -Headers $h2 `
  -ContentType "application/json" -Body $b2 | Format-List
```

EXPECTED RESULT

```
A hold object (hold_id, seat_ids, amount_cents, expires_at), same shape as Step 5.
```

Part 3: Check the reports and the database

Step 14: Correctness report

The server checks its own data. All three invariants must be true.

RUN THIS (PowerShell)

```
Invoke-RestMethod "$u/admin/stats/1" | ConvertTo-Json -Depth 5
```

EXPECTED RESULT

```
{ "seats": { "available": 1999, "sold": 1 },
  "orders_by_status": { "CONFIRMED": 1 },
  "confirmed_orders": 1, "confirmed_seats": 1,
  "outbox_backlog": 0, "notifications": 1,
  "invariants": { "no_oversell_sold_eq_confirmed_seats": true,
                  "no_seat_in_two_orders": true,
                  "payments_reconcile": true } }
(counts depend on how many times you tested; the three values must all be true)
```

If it does not match: Any false value is a real bug. Send me the full output.

Step 15: Metrics

Prometheus counters for this running instance (they reset when the service restarts).

RUN THIS (PowerShell)

```
(Invoke-WebRequest "$u/metrics" -UseBasicParsing).Content -split "`n" |
  Select-String "tm_holds_total|tm_orders_total"
```

EXPECTED RESULT

```
tm_holds_total{result="ok"} 2.0
tm_holds_total{result="conflict"} 1.0
tm_holds_total{result="not_admitted"} 1.0
tm_orders_total{status="CONFIRMED"} 1.0
```

Step 16: Look inside Neon

Open the Neon console, choose your project, open SQL Editor and run these. Wait about 10 seconds after your order so the notification arrives (the simulated email sender fails 15% of the time and retries after a few seconds).

RUN THIS (Neon SQL Editor)

```
select status, count(*) from seats where event_id = 1 group by status;
select status, failure_reason from orders order by created_at desc limit 5;
select count(*) as unpublished from outbox where published_at is null;
select kind, body from notifications order by id desc limit 5;
```

EXPECTED RESULT

```
seats:          SOLD 1 / AVAILABLE 1999   (plus any HELD seats still inside their 2 minutes)
orders:         CONFIRMED (or FAILED payment_declined)
unpublished:    0
notifications:  order.confirmed  |  order <id> order.confirmed
```

If it does not match: unpublished above 0 for a long time means the outbox loop is not running. No notification row means the consumer has not delivered yet; wait and re-run.

Part 4: Automated tests and the flash-sale load test

Run these from the project folder on your PC. Python 3.10 or newer is required (no Docker needed).

Step 17: One-time Python setup

Creates a virtual environment and installs what the tests need.

RUN THIS (PowerShell)

```
cd $HOME\projects\seatrush
python -m venv .venv
Set-ExecutionPolicy -Scope Process Bypass
.\.venv\Scripts\Activate.ps1
pip install httpx pytest
```

EXPECTED RESULT

```
The prompt now starts with (.venv).
pip prints "Successfully installed httpx-... pytest-..."
```

If it does not match: 'python' not recognised: install Python from python.org and tick 'Add to PATH'. If the activate script is blocked, the Set-ExecutionPolicy line above fixes it for this window.

Step 18: Integration tests against your live site

Four tests, including 50 users racing for a single seat (exactly one must win).

RUN THIS (PowerShell)

```
$env:BASE_URL = $u
python -m pytest -q tests
```

EXPECTED RESULT

```
....                                                     [100%]
4 passed in 12.34s          (time varies; the free tier is slow)
```

If it does not match: 'not enough free seats' means the data is used up; reset it (Part 5). A failure on the 50-user race test shows the codes it received, send them to me.

Step 19: Flash-sale load test

Simulates users going through the waiting room, holding, and paying at the same time, then checks the books. Use a modest number on the free tier.

RUN THIS (PowerShell)

```
Invoke-RestMethod "$u/health"        # wake the service first
python loadtest\flash_sale.py --base-url $u --event 2 --users 200 --concurrency 30
```

EXPECTED RESULT

```
event 'Mega Tour Flash Sale' seats=2000 users=200 waiting_room=True

=== flash sale finished in 45.2s ===
users bought: 200  hold conflicts: 18  order failures: 22  sold-out exits: 0
waiting room wait  p50=2.3s  max=14.6s
endpoint        n   p50 ms   p95 ms   p99 ms
hold          ...
status codes: {"hold:201": ..., "hold:409": ..., "order:201": ..., ...}
server stats: {"seats": {...}, "orders_by_status": {"CONFIRMED": 200, "FAILED": ...}, ...}
invariants: {"no_oversell_sold_eq_confirmed_seats": true,
             "no_seat_in_two_orders": true, "payments_reconcile": true}
RESULT: PASS - zero oversells, ledger reconciles
(numbers are an example; times will be larger on the free tier. The last line is what matters.)
```

If it does not match: RESULT: FAIL means an invariant was false: send me the output. Very slow or timeouts: the free instance has 0.1 CPU, try --users 100 --concurrency 15.

Step 20: Optional: see the rate limiter (PowerShell 7 only)

Fires 300 requests in parallel with one user id. The limit is 30 requests per second with a burst of 60.

RUN THIS (PowerShell)

```
$codes = 1..300 | ForEach-Object -Parallel {
  try { (Invoke-WebRequest "$using:u/events" -Headers @{"x-user-id"="spammer"} -UseBasicParsing).StatusCode }
  catch { [int]$_.Exception.Response.StatusCode } } -ThrottleLimit 50
$codes | Group-Object | Select-Object Name, Count
```

EXPECTED RESULT

```
Name Count
---- -----
200    ~100-250
429    ~50-200        (some 429 responses must appear)
```

If it does not match: Only 200s: your connection or the server was too slow to exceed 30 per second, which is fine. Windows PowerShell 5.1 does not support -Parallel; skip this step or install PowerShell 7.

Part 5: Reset the data

Do this when seats are used up and tests complain 'not enough free seats'.

Step 21: Reset on Neon + Render

Run the SQL in the Neon SQL Editor, then restart the Render service so it reseeds itself. Optionally clear Redis from the Render dashboard (Key Value, Connect, redis-cli, FLUSHALL).

RUN THIS (Neon SQL Editor)

```
TRUNCATE notifications, outbox, orders, holds, seats, events RESTART IDENTITY CASCADE;
```

EXPECTED RESULT

```
TRUNCATE TABLE completed.
After the restart (give it a minute): /events/1/availability shows {"available":2000,"held":0,"sold":0}
```

Quick troubleshooting

Status codes used by the API: 200 ok, 201 created, 403 not admitted from the waiting room, 404 not found, 409 seat unavailable / conflict, 410 hold expired, 429 rate limited, 503 overloaded.

What you see	Most likely cause	Fix
Not Found at the site root or at /docs/health	Wrong address	Use /health and /events at the root; /docs for the UI
First request takes ~1 minute	Free tier was asleep	Wait; call /health before demos
500 on /events	Neon connection failed	Check DATABASE_URL and DIRECT_DATABASE_URL in Render
500 on seats or holds	Redis URL missing or wrong	Check REDIS_URL; Logs may say 'Too many connections'
Never admitted to the waiting room	Background loops not running	Wake the service; check Logs; confirm RUN_WORKER_IN_API=1
429 Too Many Requests	Rate limiter (30 req/s per user)	Slow down; expected under heavy tests
503 Service Unavailable	Load shedding or deploy in progress	Retry after the Retry-After seconds
410 hold expired	More than 2 minutes between hold and order	Create a new hold
FAILED payment_declined	Simulated 10% decline	Normal; hold again and retry
'not enough free seats'	Seats sold out from earlier tests	Reset the data (Part 5)